แบบฝึกหัดปฏิบัติการคาบที่ 6: Inheritance

คำสั่ง

1. ให้ศึกษาการสืบทอดข้อมูล Inheritance จากตัวอย่างต่อไปนี้

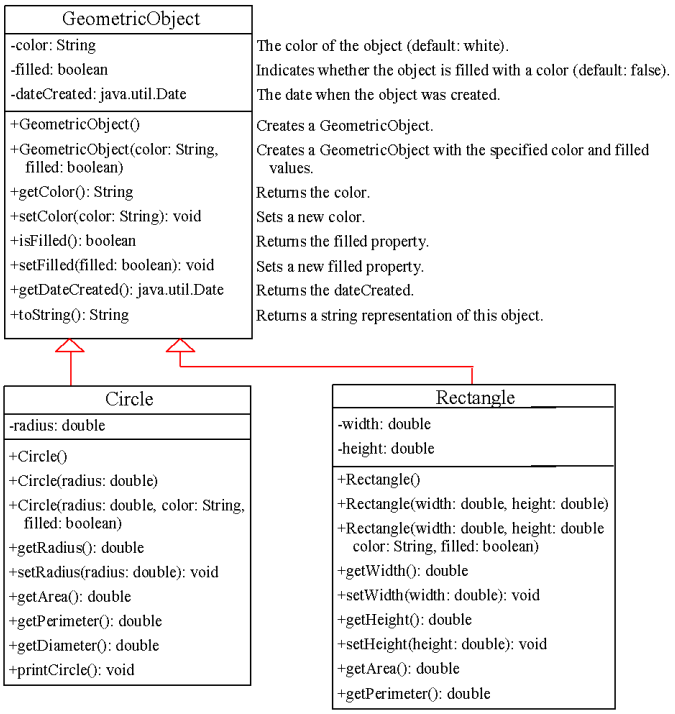
การเขียนโปรแกรมเชิงวัตถุ จะอนุญาตให้สามารถสร้างคลาสใหม่จากคลาสเดิมที่เคยมีอยู่แล้วซึ่งเรียกเทคนิคนี้ว่าการสืบทอด (Inheritance) การสืบทอด (Inheritance) เป็นเทคนิคที่สำคัญมากสำหรับการนำซอฟต์แวร์กลับมาใช้ใหม่ (Reuse software)

Super class และ Sub class

เทคนิคการสืบทอดข้อมูลอนุญาตให้กำหนดคลาสทั่วไปที่เรียกว่า General class หรือ Super class และภายหลังสามารถขยาย (extends) คลาสดังกล่าวไปเป็นคลาสที่มีความจำเพาะเจาะจงได้ specialized class หรือ sub class เราสามารถใช้คลาสเพื่อจำลองวัตถุที่มีชนิดข้อมูลเดียวกันได้ คลาสต่าง ๆ กันอาจจะมีคุณสมบัติและพฤติกรรมที่เหมือนกัน ที่สามารถทำให้อยู่ในรูปทั่วไปได้และสามารถใช้ร่วมกับคลาสอื่น ๆ ได้ เรายังสามารถกำหนดคลาสที่มีคุณสมบัติจำเพาะ ที่สืบทอดหรือขยายมาจากคลาสทั่วไปหรือ Generalized class ได้โดยที่คลาสที่มีความจำเพาะเหล่านั้นสืบทอดคุณสมบัติและเมธอดมาจากคลาสทั่วไป

ตัวอย่าง

สมมติต้องการคลาสวงกลม สี่เหลี่ยมและสามเหลี่ยม คลาสต่าง ๆ เหล่านี้จะมีคุณลักษณะบางอย่างที่คล้ายกัน การออกแบบคลาสเพื่อไม่ให้ใช้ตัวแปรซ้ำ และทำให้ง่ายต่อการแก้ไข วิธีที่ดีที่สุดคือการใช้หลักการของการสืบทอดข้อมูล โดยการสร้างคลาสที่คลาสทุกคลาสมีคุณสมบัติเดียวกันเรียกว่า Superclass ซึ่งจากตัวอย่างคือ GeometricObject หลังจากนั้นเมื่อต้องการสร้างวงกลม Circle ก็ให้สืบทอดคุณสมบัติจากคลาส Geometric และเพิ่มคุณสมบัติเฉพาะตัวของวงกลมเข้าไป



1. ให้ศึกษาและทดลองพิมพ์ Code ตัวอย่างการสืบทอดคลาสจากตัวอย่างต่อไปนี้

```
public abstract class GeometricObject {
	private String color = "white";
	private boolean filled;
	/** Default constructor */
	protected GeometricObject() {
	}
	/** Convenience constructor */
	protected GeometricObject(String color, boolean filled) {
		this.color = color;
		this.filled = filled;
	}
	public String getColor() {
		return color;
	}
	public void setColor(String color) {
		this.color = color;
	}
	public boolean isFilled() {
		return filled;
	}
	public void setFilled(boolean filled) {
		this.filled = filled;
```

```
	}
	public abstract double findArea();
	public abstract double findPerimeter();
}
public class Circle extends GeometricObject {
	private double radius;

	/** Default constructor */
	public Circle() {
		this(1.0);
	}

	/** Radius convenience constructor */
	public Circle(double radius) {
		this(radius, "white", false);
	}

	/** Convenience constructor for all properties */
	public Circle(double radius, String color, boolean filled) {
		super(color, filled);
		this.radius = radius;
	}

	/**
	 * Return the radius
	 * @return radius Current radius of Circle
	 */
	public double getRadius() {
		return radius;
	}

	/**
	 * Set the radius of the circle
	 */
	public void setRadius(double radius) {
		this.radius = radius;
	}

	/**
	 * Returns the area of the current circle
	 * Implemention of abstract method in GeometricObject
	 * @return area of the circle
	 */
	public double findArea() {
		return radius * radius * Math.PI;
	}

	/**
	 * Returns the perimeter of the current circle
	 * Implemention of abstract method in GeometricObject
	 * @return perimeter of the circle
	 */
	public double findPerimeter() {
		return 2 * radius * Math.PI;
	}

	/**
	 * Provide a string representation of the object
```

```
     */
    public String toString() {
        return "Circle: radius = " + radius;
    }
}
```

1.1 **ให้อธิบายลำดับการเรียกใช้** constructor **เมื่อสร้างวัตถุจากคลาส** Circle

เมื่อสั่ง `new Circle(radius, color, filled)` **จะเริ่มทำงานที่** Constructor **ของ** `Circle`

ในบรรทัดแรกของ Constructor **มีการเรียก** `super(color, filled)` **ทำให้โปรแกรมสลับไปทำงานที่** Constructor **ของ** Superclass (GeometricObject) **ก่อนเพื่อกำหนดค่า** `color` **และ** `filled`

เมื่อ `GeometricObject` **ทำงานเสร็จ จึงกลับมากำหนดค่า** `this.radius = radius` **ใน** Subclass (`Circle`)

1.2 **ให้สร้างคลาส** Rectangle **ตาม** UML Class diagram **ด้านบน**

```
public class Rectangle extends GeometricObject {
    private double width;
    private double height;

    public Rectangle() {
        this(1.0, 1.0);
    }

    public Rectangle(double width, double height) {
        super("white", false);
        this.width = width;
        this.height = height;
    }

    public Rectangle(double width, double height, String color,
boolean filled) {
        super(color, filled);
        this.width = width;
        this.height = height;
    }

    public double getWidth() { return width; }
    public void setWidth(double width) { this.width = width; }

    public double getHeight() { return height; }
    public void setHeight(double height) { this.height = height; }

    @Override
    public double findArea() {
        return width * height;
    }
```

```
    @Override
    public double findPerimeter() {
        return 2 * (width + height);
    }
}
```

2. จงหาข้อผิดพลาดของส่วนของโปรแกรมต่อไปนี้

```
public class Circle{
   private double radius;

   public Circle(double radius){
      radius=radius;
   }
   public double getRadius(){
      return radius;
   }
   public double findArea(){
      return radius*radius*Math.PI;
   }
}
class Cylinder extends Circle{
   private double length;

   Cylinder(double radius, double length){
      Circle(radius);
      length=length;
   }
}
```

ในคลาส `Circle`: `radius = radius;` **ไม่ได้กำหนดค่าเข้าตัวแปรของวัตถุ เพราะชื่อพารามิเตอร์ซ้ำกับอินสแตนซ์ (Shadowing) ต้องแก้ไขเป็น** `this.radius = radius;`

ในคลาส `Cylinder`: **คำสั่ง** `Circle(radius);` **ผิดหลักไวยากรณ์ภาษา Java ในการเรียก Constructor ของคลาสแม่ ต้องใช้คำสั่ง** `super(radius);` **และต้องวางไว้ในบรรทัดแรกของ Constructor**

ในคลาส `Cylinder`: `length = length;` **ต้องแก้ไขเป็น** `this.length = length;`

3. จงหาผลลัพธ์การรันโปรแกรมต่อไปนี้

3.1

```
class A{
   public A(){
      System.out.println("The no-arg constructor of A is
invoked");
   }
}
class B extends A{
   public B(){

   }
}
public class C{
   public static void main(String[] args){
     B b =new B();
   }
}
```

ผลลัพธ์การรันโปรแกรม

```
The no-arg constructor
of A is invoked
```

3.2

Animal.java:

```
01     public class Animal {
02         public Animal() {
03             System.out.println("A new
animal has been created!");
04         }
05
06         public void sleep() {
07             System.out.println("An animal
sleeps...");
08         }
09
10         public void eat() {
11             System.out.println("An animal
eats...");
12         }
13     }
```

Bird.java:

```
01     public class Bird extends Animal {
02         public Bird() {
03             super();
04             System.out.println("A new
bird has been created!");
05         }
06
07         @Override
08         public void sleep() {
09             System.out.println("A bird
sleeps...");
10         }
11
12         @Override
13         public void eat() {
14             System.out.println("A bird
eats...");
15         }
16     }
```

Dog.java:

```
01     public class Dog extends Animal {
02         public Dog() {
03             super();
04             System.out.println("A new dog
has been created!");
05         }
```

MainClass.java:

```
01     public class MainClass {
02         public static void main(String[]
args) {
03             Animal animal = new Animal();
04             Bird bird = new Bird();
05             Dog dog = new Dog();
```

```
06
07          @Override
08          public void sleep() {
09              System.out.println("A dog
sleeps...");
10          }
11
12          @Override
13          public void eat() {
14              System.out.println("A dog
eats...");
15          }
16      }
```

```
06
07              System.out.println();
08
09              animal.sleep();
10              animal.eat();
11
12              bird.sleep();
13              bird.eat();
14
15              dog.sleep();
16              dog.eat();
17          }
18      }
```

ผลลัพธ์การรันโปรแกรม

```
A new animal has been created!

A new animal has been created!

A new bird has been created!

A new animal has been created!

A new dog has been created!



An animal sleeps...

An animal eats...

A bird sleeps...

A bird eats...

A dog sleeps...

A dog eats...
```

4. จากโปรแกรมด้านล่างต่อไปนี้ ข้อใดคือ Overriding ที่สามารถนำมาเติมลงในบรรทัดที่ 9 ของโปรแกรมได้โดยไม่ทำให้เกิดข้อผิดพลาดขึ้นกับการทำงานของโปรแกรม อธิบายเหตุผลประกอบ

```
1  class SuperClass{
2     private int num=1;
3     protected int getNumber(){
4        return num;
5     }
6  }
7  class Subclass extends SuperClass{
8     private int num=10;
9     //overiding method
10    public static void main(String[] args){
11      Subclass s= new Subclass();
12      System.out.println(s.getNumber());
13    }
14 }
```

โปรแกรม	สามารถนำมาเติมลงในบรรทัดที่ 9 ของโปรแกรมได้โดยไม่ทำให้เกิดข้อผิดพลาดหรือไม่	อธิบายเหตุผลประกอบ
`protected int getNumbers(){ return num+5; }`	**ไม่เกิดข้อผิดพลาด**	ไม่ใช่ Overriding แต่เป็น Method ใหม่เพราะชื่อเมธอดต่างกัน (`getNumbers`)
`protected long getNumber(){ return num+5; }`	**เกิดข้อผิดพลาด**	Return type ต่างกัน (`long` ไม่ตรงกับ `int` ของเมธอดเดิม)
`protected int getNumber(){ return num+5; }`	**ไม่เกิดข้อผิดพลาด**	เป็น Valid Overriding ถูกต้องตามกฎทุกประการ
`public int getNumber(){ return num+5; }`	**ไม่เกิดข้อผิดพลาด**	ป็น Valid Overriding (สามารถขยายระดับการเข้าถึงจาก `protected` เป็น `public` ได้)
`protected int getNumber(int num){ return num+5; }`	**ไม่เกิดข้อผิดพลาด**	ไม่ใช่ Overriding แต่เป็น Method Overloading เพราะมีพารามิเตอร์ต่างกัน
`int getNumber(){ return num+5; }`	**เกิดข้อผิดพลาด**	ลดระดับ Access Modifier จาก `protected` ลงมาเป็น Default (Package-private) ไม่ได้
`private int getNumber(){ return num+5; }`	**เกิดข้อผิดพลาด**	ลดระดับ Access Modifier จาก `protected` ลงมาเป็น `private` ไม่ได้